DAY 16 | HANDS-ON ACTIVITIES

Data Pipelines, Spark and PySpark: Problem Statements

Seven problems building PySpark pipelines and SQL. In Colab, Spark is ready after a quick install. In VS Code, install Java 17 and pyspark first.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Hello Spark

Level: Warm-up | **Concepts:** SparkSession, createDataFrame, show

Start Spark, create a DataFrame of 5 cities with population and country, print the schema, show the rows and count them.

Sample run / expected result

```
root
 |-- city: string
 |-- population: long
...
Count: 5
```

Hints

- spark.createDataFrame(list_of_tuples, ["city", "population", "country"])

Problem 2: Filter, Derive and Stay Lazy

Level: Easy | **Concepts:** filter, withColumn, when, laziness

From the orders data, keep amount > 0, trim city names, and add a size column: large (>= 1000), medium (>= 300) or small. Time the transformations and then count(); explain the difference.

Sample run / expected result

```
|order_id|city |amount|size |
|1002    |Tokyo|1200.0|large|
Transformations: 0.01 s | count(): 1.8 s
```

Hints

- Chain F.when(...).when(...).otherwise(...).
- time.perf_counter()

Problem 3: Join Detective

Level: Medium | **Concepts:** inner, left, left_anti

Join orders with customers using inner and left joins and compare row counts. Use left_anti to list orders whose customer is missing.

Sample run / expected result

```
orders: 10 | inner: 8 | left: 10
Orders with unknown customer: 1009, 1010
```

Hints

- Count before and after every join.

Problem 4: CTE and Window in Spark SQL

Level: Medium | **Concepts:** WITH, RANK() OVER, temp views

Register orders as a temp view. Using a CTE for daily city revenue, rank cities within each day and return only rank 1 per day. Confirm the result matches the same query in SQLite.

Sample run / expected result

```
2026-11-12 | Tokyo     | 4500.0
2026-11-13 | Bengaluru | 3100.0
Spark result == SQLite result: True
```

Hints

- RANK() OVER (PARTITION BY order_date ORDER BY revenue DESC)

Problem 5: Medallion Pipeline

Level: Challenging | **Concepts:** bronze, silver, gold, Parquet

Write raw data as bronze Parquet, clean it into silver, join and aggregate into gold daily revenue by city, partitioned by date. Re-run and show counts are unchanged.

Sample run / expected result

```
bronze: 12 | silver: 9 | gold: 7
Re-run: same counts
```

Hints

- mode("overwrite") makes re-runs idempotent.

Problem 6: Verify a Codex Transformation NEW (v4)

Level: Challenging | **Concepts:** AI-assisted DE, golden examples

Use the Codex prompt from the deck (or the notebook's online cell) to generate sla_metrics(tickets). Build a golden example of 6 tickets with a hand-calculated answer. Run the generated function, compare, and list every difference.

Then check one number a second way (pandas or SQL) and write a short verdict: keep, fix or reject.

Sample run / expected result

```
golden: high 1/2 met (50.0%), medium 2/2 (100.0%), low 0/1 (0.0%)
generated: high 1/3 met (33.3%)  <- duplicate ticket counted twice
verdict: fix (add dropDuplicates)
```

Hints

- Include a duplicate, an unresolved ticket and a ticket exactly on the SLA boundary.
- Note the prompt and roughly how many tokens it used.

Problem 7: Parameterised Pipeline with Quality Gates

Level: Stretch | **Concepts:** run date, checks, run log, PyTest

Write run(run_date) that processes one day through silver and gold, checks not-empty, unique keys and reconciliation, overwrites the gold partition and appends to a run log. Write PyTest tests for each check.

Sample run / expected result

```
run("2026-11-13") -> bronze 3, silver 3, gold 3
Re-run -> same counts, log has 2 rows
4 passed
```

Hints

- class PipelineError(Exception): pass
- Use a session-scoped local[1] SparkSession fixture.